

PDF Toolbox (Foliodex)

Application Functional Specification & QA Test Manual

Version: 1.0.1 **Date:** 2026-10-05

Table of Contents

1. [Application Overview](#)
 2. [Setup and Test Environment](#)
 3. [Navigation and Screen-by-Screen Documentation](#)
 4. [Detailed Functionality Reference](#)
 5. [End-to-End Business Workflows](#)
 6. [QA Test Manual - Strategy](#)
 7. [Individual Test Cases](#)
 8. [End-to-End Test Scenarios](#)
 9. [Regression Checklist](#)
 10. [Defect Reporting Template](#)
 11. [QA Execution Summary](#)
 12. [Technical Reference and Troubleshooting](#)
-

1. Application Overview

Purpose & Business Objectives: PDF Toolbox (Foliodex) provides an all-in-one, completely local mobile suite to inspect, manipulate, annotate, secure, and convert PDF documents on Android devices without relying on third-party servers.

Target Users & Roles: Professionals, students, mobile workers, and privacy-conscious users handling sensitive documents. There is only a single user role (the device owner), as the app operates completely offline.

Main Capabilities:

- View PDFs with zoom/scroll.
- Merge, split, compress, and rotate PDFs.

- Convert images to PDF.
- Edit, annotate, and watermark PDFs.
- Encrypt and decrypt PDFs.

Supported Platforms: Android 7.0 (API 24) to Android 15 (API 34+).

2. Setup and Test Environment

Required Software:

- Android Studio Hedgehog (2023.1.1) or newer.
- Android Device or Emulator running Android 7.0+.
- ADB (Android Debug Bridge).

Installation Instructions:

1. Clone the repository.
2. Build debug APK using `./gradlew assembleDebug`.
3. Install via `adb install -r app/build/outputs/apk/debug/app-debug.apk`.

Environment Reset: Clear app data to reset test state: `adb shell pm clear com.pdftoolbox.app`

3. Navigation and Screen-by-Screen Documentation

Home Screen

- **Access:** Default screen on launch (`R.id.navigation_home`).
- **Functionality:** Displays a list of Recent Files (up to 30 items) with timestamps. A Floating Action Button (FAB) allows selecting a new PDF from device storage.
- **Interactions:** Tapping a recent file opens the PDF Viewer.

Tools Screen

- **Access:** Bottom Navigation -> "Tools" (`R.id.navigation_tools`).
- **Functionality:** A grid/list of 14 PDF manipulation tools.
- **Interactions:** Tapping a tool opens the respective Tool configuration screen.

Settings Screen

- **Access:** Bottom Navigation -> "Settings" (R.id.navigation_settings).
- **Functionality:** Manage app configuration, primarily the DayNight theme (Light, Dark, System Default).

PDF Viewer Screen

- **Access:** From Home Screen by selecting a PDF.
 - **Functionality:** Full-screen multi-page rendering using native PdfRenderer. Memory managed via LRU Cache.
 - **Controls:** Scroll up/down to view pages.
-

4. Detailed Functionality Reference

FEAT-001: PDF Viewer

- **Business Purpose:** Fast, reliable rendering of PDF documents.
- **Preconditions:** A PDF exists on the device.
- **Edge Cases:** Large PDFs (>500MB) or PDFs with complex forms. Handles OutOfMemory via LruCache.

FEAT-003: Merge PDF

- **Business Purpose:** Combine multiple PDF files.
- **Usage:** Tools -> Merge -> Select multiple PDFs -> Drag to reorder -> Tap "Merge".
- **Expected Result:** A new PDF combining all selected documents in order.

FEAT-013: Protect PDF

- **Business Purpose:** Encrypt documents for security.
- **Usage:** Tools -> Protect -> Select PDF -> Enter 128-bit password -> Save.
- **Validation:** Password cannot be empty.

(Reference FEATURE_INVENTORY.md for the full list of 17 features)

5. End-to-End Business Workflows

Workflow 1: Prepare and Secure a Document

1. User creates a document from images (Convert Images to PDF).
 2. User opens the generated PDF in the Viewer to verify pages.
 3. User navigates to Tools -> Protect PDF.
 4. User encrypts the generated PDF.
 5. User verifies encryption by attempting to open the file (prompts for password).
-

6. QA Test Manual - Strategy

Scope: UI testing, functional testing of PDF operations, performance on large files, and permissions validation. **Priority:**

- **P0:** Critical core (Viewing, Merging, Saving, Crashing).
 - **P1:** Main manipulations (Split, Protect, Compress).
 - **P2:** UI and state (Recent files, Themes).
 - **P3:** Edge cases.
-

7. Individual Test Cases

(See QA_TEST_CASES.md for full tabular format)

TC-001 [P0] View multi-page PDF

- **Steps:** Launch App -> Tap FAB -> Select multi-page PDF.
- **Expected:** PDF renders immediately, scrolling is smooth, no crashes.

TC-002 [P1] Merge two PDFs

- **Steps:** Tools -> Merge -> Select two PDFs -> Reorder -> Save.
 - **Expected:** Resulting file contains pages from both in correct order.
-

8. End-to-End Test Scenarios

Scenario 1: Extract, Edit, and Protect

1. Extract images from DocumentA.pdf.
2. Convert extracted images back into a new DocumentB.pdf.
3. Annotate DocumentB.pdf using Edit & Annotate tool.
4. Protect DocumentB.pdf with a password.

5. **Expected:** All operations succeed without errors. DocumentB.pdf requires password to view.
-

9. Regression Checklist

- ☐ View a 100+ page PDF (Verify LRU Cache).
 - ☐ Convert images to PDF.
 - ☐ Split a PDF (Verify valid ranges like "1-3,5").
 - ☐ Encrypt and Unlock a PDF.
 - ☐ Force close app and check Recent Files.
 - ☐ Switch Dark/Light mode.
-

10. Defect Reporting Template

Defect ID: BUG-[000] **Summary:** [Short description] **Feature/TC ID:** [e.g., FEAT-004 / TC-003] **Environment:** [Android OS Version, Device Model] **Steps to Reproduce:** 1. 2. **Expected Result:** [] **Actual Result:** [] **Severity:** [High/Medium/Low] **Evidence:** [Attach screenshot/logcat]

11. QA Execution Summary

- Total Test Cases: 8
- Passed: 0
- Failed: 0
- Blocked: 0
- Not Run: 8
- Pass Percentage: 0%

(Metrics pending execution)

12. Technical Reference and Troubleshooting

Architecture: Single-activity MVVM pattern with PdfToolsRepository handling Apache PDFBox operations on Dispatchers.IO.

Troubleshooting:

- **OutOfMemoryError:** Check LruCache sizing in ViewerFragment.
- **IllegalStateException: Already has open page:** Ensure renderMutex.withLock is used in coroutines before operating on PdfRenderer.
- **Permission Denied:** Ensure SAF takePersistableUriPermission is called immediately on ACTION_OPEN_DOCUMENT result.